

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 13. Dictionary

Lecture: 137. Dictionary - Introduction

Section 1: Lecture Summary

This lecture introduces the Python dictionary data type, a powerful associative structure distinct from strings, lists, tuples, and sets. Dictionaries store data as key-value pairs, where each key maps uniquely to a value. The lecture covers dictionary creation, accessing values via keys, updating existing entries, adding new key-value pairs, and traversing dictionaries using loops. It emphasizes that keys must be immutable (e.g., integers, strings, or tuples), while values can be of any type, including complex types such as lists, tuples, sets, or even other dictionaries. The lecture also demonstrates common use cases like storing words and their meanings or roll numbers with student names.

Section 2: Key Concepts and Explanations

- Dictionary Overview

- A dictionary is an associative collection of key-value pairs.
- Syntax uses curly braces `{}`, with elements as `'key: value'`.
- Example: `{1: "one", 2: "two"}` contains two entries.
- Each entry or item consists of a key and a value.
- Keys are unique within a dictionary; values can repeat.
- Efficient for searching values by their keys.

- Creating Dictionaries

- Empty dictionary: `D = {}` creates an empty dictionary, not a set.
- Populate with entries using colon-separated pairs inside braces.

- Accessing Values

- Use square brackets with the key: `D[key]` returns the associated value.
- If the key does not exist, an error (`KeyError`) occurs.

- Updating and Adding Entries

- Assigning a new value to an existing key updates the entry:

`D[key] = new_value` modifies the value for that key.`

- Assigning a value to a new key adds that key-value pair:

`D[new_key] = value` inserts a new entry.`

- Traversing Dictionaries

- Iterating over a dictionary using `for key in D:` loops over keys only.`

- To access values inside the loop: `D[key]` returns the value.`

- Can print both keys and values during traversal.

- Data Types for Keys and Values

- Keys must be immutable types because dictionary uses hashing.

- Allowed key types: integers, floats, strings, tuples (immutable sequences).

- Not allowed as key types: lists, sets, dictionaries (mutable types).

- Values can be any data type, including complex collections like lists, tuples, sets, or nested dictionaries.

- Dictionary Use Cases

- Storing word-to-meaning pairs (like a real dictionary).

- Mapping roll numbers to student names (key-value pairs).

- Advantageous for organizing large and complex data sets due to quick lookup by keys.

Section 3: Example Code and Use Cases

Creating a dictionary with integer keys and string values

```
D1 = {1: "one", 2: "two", 3: "three", 4: "four"}
print(D1)
```

Creates a dictionary with 4 entries mapping integers to their word form strings.

Accessing value by key

Attempting to access a non-existing key results in KeyError

`print(D1[5])` Uncommenting will raise KeyError`

```
print(D1[1]) # Output: one
```

Access shows retrieval of values by keys; accessing missing keys raises error.

Updating existing key's value

```
D1[3] = "tres"
print(D1)
```

Changes value associated with key 3.

Adding a new key-value pair

```
D1[5] = "five"
print(D1)
```

Inserts new entry with key 5.

Traversing a dictionary (printing keys and values)

```
for key in D1:
    print(f"Key: {key}, Value: {D1[key]}")
```

Loop shows how to access all keys and values.

Dictionary with mixed type keys and values

```
D2 = {
    1: 3.5,
    2.4: True,
    (1+2j): "complex key"
}
print(D2)
```

Demonstrates keys of different immutable types (int, float, complex) and mixed values.

Dictionary values as collections

```
D3 = {
    1: [1, 2, 3],          # list as a value
    2: (4, 5, 6),          # tuple as a value
    3: {7, 8, 9},          # set as a value
    4: {1: 1, 2: 2}        # dictionary as a value
}
print(D3)
```

Shows values can be lists, tuples, sets, or even dictionaries.

Using tuple as a key (allowed)

Attempting to use list as a key (not allowed)

D5 = {[1, 2]: "list as key"} Raises TypeError: unhashable type: 'list'

```
D4 = {(1, 2): "tuple as key"}
print(D4)
```

Tuples are allowed as keys, lists are not due to mutability.

Example dictionary simulating real dictionary words and meanings

```
dictionary = {
    "abacus": "a calculator",
    "bachelor": "unmarried person",
    "cable": "strong rope"
}
print(dictionary["abacus"])
```

Emulates a real-world dictionary with words as keys and definitions as values.

Student roll number and names

```
students = {
    1: "John",
    2: "Smith",
    3: "Alice"
}
print(students[1])
```

Maps student roll numbers to their names.

Section 4: Key Takeaways

- Dictionary stores data as key-value pairs with unique keys enabling fast lookup.
- Created with curly braces `{}`, with entries as `key: value`.
- Keys must be immutable types such as integers, strings, floats, or tuples.
- Values can be any type, including complex collections or other dictionaries.
- Accessing a value by a non-existent key raises a `KeyError`.
- Assigning a value to an existing key updates it; assigning to a new key inserts a new pair.
- Iterating over a dictionary yields keys; values can be accessed inside the loop using the keys.

- Dictionaries are highly flexible for organizing and managing complex data efficiently.